



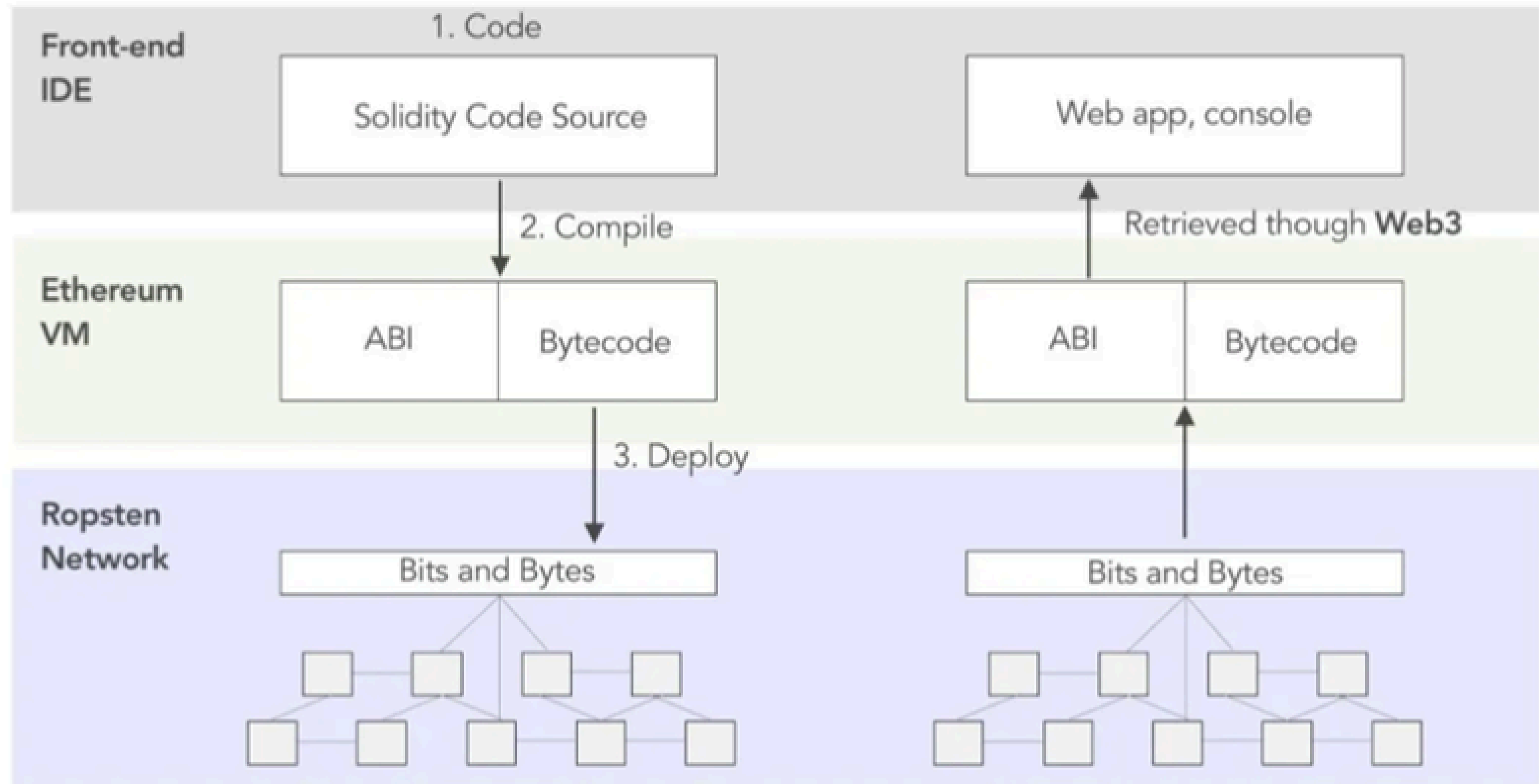
BLOCKCHAIN TECHNOLOGY



ethereum

Writing contracts TO Ethereum

Reading contracts FROM Ethereum



CALLING & SENDING FUNCTION

Calling function

- 1) No data recorded on the blockchain
- 2) Return data
- 3) Execute immediately
- 4) Execute at no cost

Sending function

- 1) Data recorded on the blockchain
- 2) Return transaction hash
- 3) Take time to execute
- 4) Execute with cost fee

$$\text{Transaction fee} = \text{Gas used} \times \text{Gas price}$$

Solidity source code

Compiler

Ethereum Virtual Machine (EVM) bytecode

Application Binary Interface (ABI)

Smart contract structure/syntax

```
pragma solidity >=0.5.0 <0.6.0;
```

```
contract HelloWorld {  
  
}
```

```
pragma solidity ^0.8.0;
```

```
contract Example {  
  
    // storage variables  
  
    // functions  
}
```

State Variables and Integers

The uint data type is an **unsigned** integer, meaning its value must be non-negative. There's also an int data type for signed integers.

contract Example {

 // declare unsigned integer variable

 uint myUnsignedInteger = 100;

}

Note: uint หมายถึง uint256 ซึ่งก็คือ 256-bit unsigned integer ในกรณีที่ต้องการระบุให้ชัดเจนว่าต้องการขนาดเป็นกี่ bit ให้ระบุหมายเลข เช่น uint8, uint16, uint32 เป็นต้น

Variables

```
// Fixed-size types
bool isDone;
uint i;
address owner;
bytes data;
```

```
// User-defined data
struct User {
    uint userId;
    string userName;
}
```

```
// Variable-size types
string name;
bytes data;
uint[] user;
mapping(address => uint) balances
```

Math Operations

1) Addition: $x + y$

2) Subtraction: $x - y$,

3) Multiplication: $x * y$

4) Division: x / y

5) Modulus / remainder: $x \% y$

ตัวอย่าง $21 \% 4$ is 1 เพราะว่า 1 คือเศษ (remainder)

6) Exponential operator (i.e. "x to the power of y", x^y):

`uint ex = 3 ** 2;` เท่ากับ 3 ยกกำลังสอง เท่ากับ 9

Type of variables

- **State Variables** – Variables whose values are permanently stored in a contract storage.
- **Local Variables** – Variables whose values are present till function is executing.
- **Global Variables** – Special variables exists in the global namespace used to get information about the blockchain.

Source: https://www.tutorialspoint.com/solidity/solidity_variables.htm

State Variables – Variables whose values are permanently stored in a contract storage.

```
pragma solidity ^0.8.0;
contract SolidityTest {
    uint storedData;    // State variable
    constructor() public {
        storedData = 10; // Using State variable
    }
}
```

Source: https://www.tutorialspoint.com/solidity/solidity_variables.htm

Local Variables – Variables whose values are present till function is executing.

```
pragma solidity ^0.8.0;
contract SolidityTest {
    uint storedData; // State variable
    constructor() public {
        storedData = 10;
    }
    function getResult() public view returns(uint){
        uint a = 1; // local variable
        uint b = 2;
        uint result = a + b;
        return result; //access the local variable
        //return storedData; //access the state variable
    }
}
```

Global variables – Variables which exist in global workspace and provide information about the blockchain and transaction properties.

Name	Returns
blockhash(uint blockNumber) returns (bytes32)	Hash of the given block - only works for 256 most recent, excluding current, blocks
block.coinbase (address payable)	Current block miner's address
block.difficulty (uint)	Current block difficulty
block.gaslimit (uint)	Current block gaslimit
block.number (uint)	Current block number
block.timestamp (uint)	Current block timestamp as seconds since unix epoch
gasleft() returns (uint256)	Remaining gas

block.timestamp (uint)	timestamp as seconds since unix epoch
gasleft() returns (uint256)	Remaining gas
msg.data (bytes calldata)	Complete calldata
msg.sender (address payable)	Sender of the message (current caller)
msg.sig (bytes4)	First four bytes of the calldata (function identifier)
msg.value (uint)	Number of wei sent with the message
now (uint)	Current block timestamp
tx.gasprice (uint)	Gas price of the transaction

Global variables – Variables which exist in global workspace and provide information about the blockchain and transaction properties.

Name	Returns
blockhash(uint blockNumber) returns (bytes32)	Hash of the given block - only works for 256 most recent, excluding current, blocks
block.coinbase (address payable)	Current block miner's address
block.difficulty (uint)	Current block difficulty
block.gaslimit (uint)	Current block gaslimit
block.number (uint)	Current block number
block.timestamp (uint)	Current block timestamp as seconds since unix epoch

gasleft() returns (uint256)	Remaining gas
msg.data (bytes calldata)	Complete calldata
msg.sender (address payable)	Sender of the message (current caller)
msg.sig (bytes4)	First four bytes of the calldata (function identifier)
msg.value (uint)	Number of wei sent with the message
now (uint)	Current block timestamp
tx.gasprice (uint)	Gas price of the transaction

Function visibility

Who can call a smart contract function?

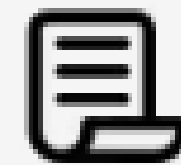
Before getting into the different types of function visibility, it's helpful to understand what the different categories of potential callers to a function are first.



MyContract



DerivedContract
is MyContract



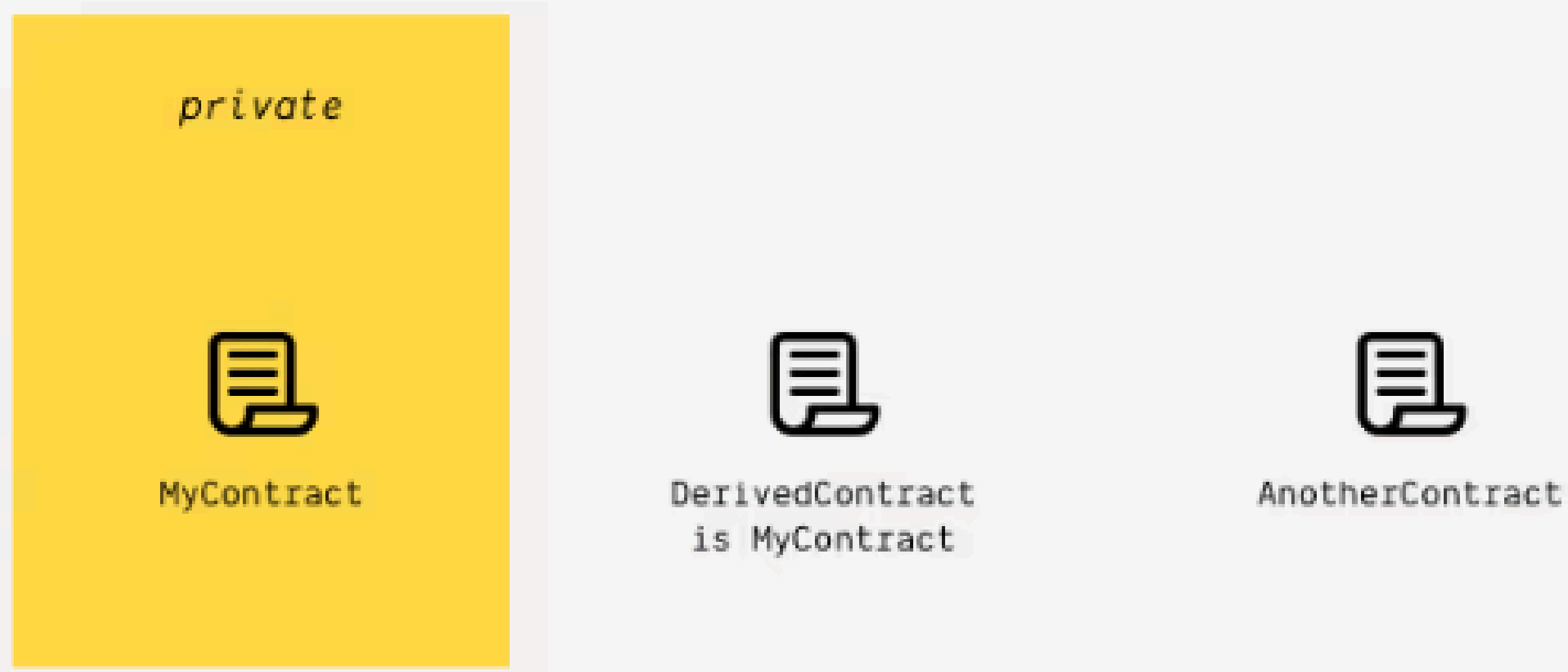
AnotherContract

private / internal / external / public

<https://bitsofco.de/solidity-function-visibility-explained/>

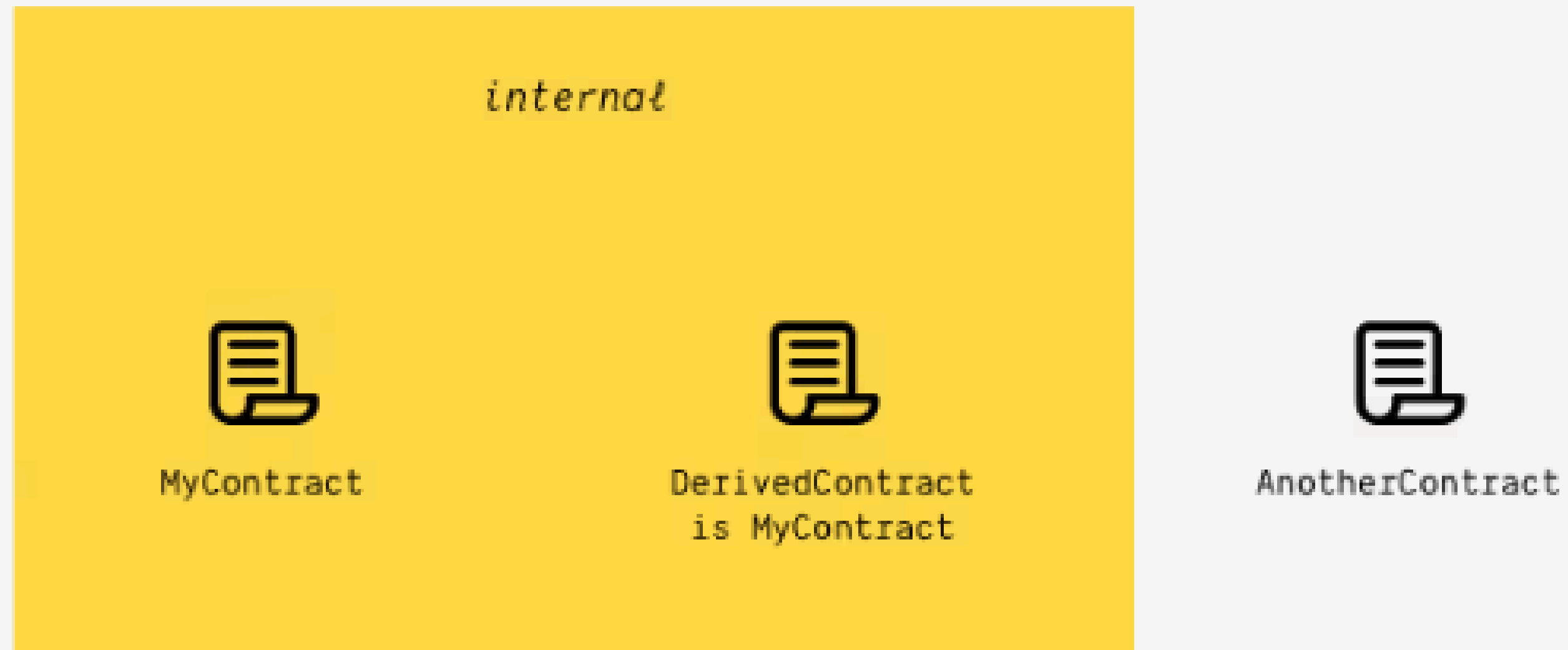
Function visibility - private

A **private** function is one that can only be called by the main contract itself. Although it's not the default, it is generally good practice to keep your functions private unless a scope with more visibility is needed.



Function visibility - internal

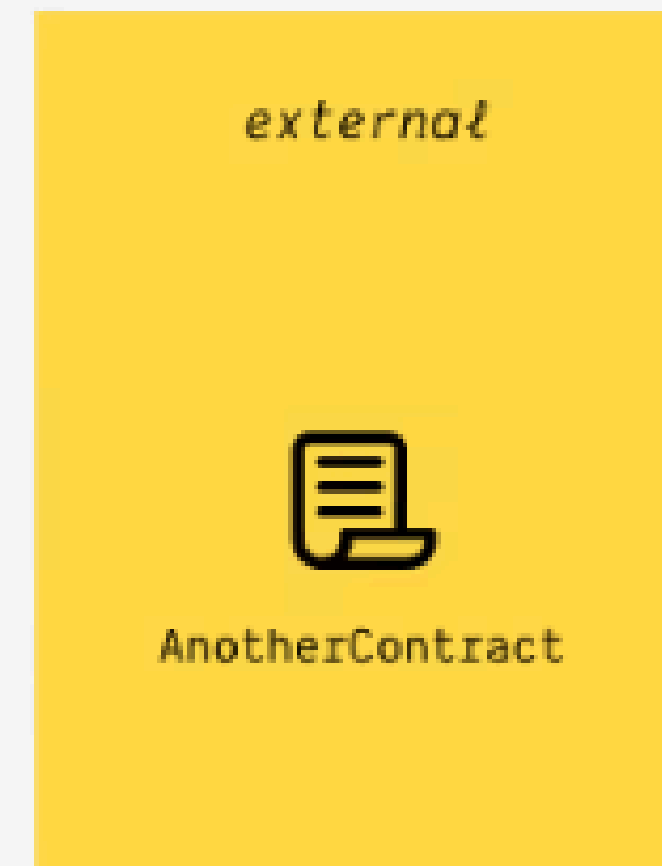
An **internal** function can be called by the main contract itself, plus any derived contracts. As with **private** functions, it's generally a good idea to keep your functions **internal** wherever possible.



Function visibility - external

An **external** function can only be called from a third party. It cannot be called from the main contract itself or any contracts derived from it.

External functions have the benefit that they can be more performant due to the fact that their arguments do not need to be copied to memory. So, where possible, it's advisable to keep logic that only needs to be accessed by an external party to an **external** function.



Function visibility - public

Finally, a **public** function can be called from all potential parties. Unless otherwise specified, all functions are made public by default.



เครื่องมือช่วยในการคำนวณค่าเงิน/หน่วยของเงิน Wei/GWei/Ether

https://eth-converter.com

Ethereum Unit Converter

Simple Unit Converter

After constantly using a calculator to convert between Ether units i thought it would neat to implemen website for everyone to use. There is a [simple](#) and [full](#) converter page, since the majority wouldn't bot

Wei

Wei

Gwei

Gwei

Ether

Ether

Already Learned

Constructor – A special function declared with constructor keyword which will be executed once per contract and is invoked when a contract is created.

State Variables – Variables per Contract to store the state of the contract.

Functions – Functions per Contract which can modify the state variables to alter the state of a contract.

Already Learned

Visibility Quantifiers

private – Private functions/ Variables can only be used internally and not even by derived contracts.

internal – Internal functions/ Variables can only be used internally or by derived contracts

external – External functions are meant to be called by other contracts. They cannot be used for internal call. To call external function within contract `this.function_name()` call is required. State variables cannot be marked as external.

public – Public functions/ Variables can be used both externally and internally. For public state variable, Solidity automatically creates a getter function.

.

Already Learned

View function

View functions ensure that they will not modify the state. A function can be declared as view.

```
contract EasyBank {  
    mapping(address => uint256) balances;  
  
    function getBalance() public view returns (uint256) {  
        balances[msg.sender] += 1;  
        return balances[msg.sender];  
    }  
}
```



Already Learned

payable

payable functions are part of what makes Solidity and Ethereum so cool — they are a special type of function that can receive Ether.

```
function deposit() public payable {  
    balances[msg.sender] += msg.value;  
}
```

Modifier

```
modifier onlyOwner {  
    require(msg.sender == owner);  
    _;  
}
```

In this example, we define a modifier called `onlyOwner` that checks if the address of the sender is equal to the owner of the contract. If the check passes, the `_` symbol indicates where the code of the modified function will be inserted. In this case, the modifier is used to restrict access to a function so that only the owner of the contract can call it.

To use a modifier in a function, you simply add the modifier's name to the function definition, like this:

Solidity

```
function withdraw() public onlyOwner {  
    // function code here  
}
```

Function Modifiers are used to modify the behaviour of a function.

For example: [Click](#)

address(this).balance

address(this).balance is used to get the balance of the contract

Workshop

EasyBank.sol

Write a smart contract with 3 functions

- 1) `getBalance()`
- 2) `deposit()`
- 3) `withdraw(uint256)`

Workshop

EasyBank.sol (cont.)

- 1) ประกาศตัวแปร balances เป็น mapping address/uint256
- 2) สร้างฟังก์ชัน getBalance แบบ public และคืนค่ายอดเงินคงเหลือของบัญชีที่เรียก smart contract นี้มา (ฟังก์ชันนี้ไม่สามารถเปลี่ยนแปลงค่าตัวแปร state ได้)
- 3) สร้างฟังก์ชัน deposit เพื่อให้สามารถรับเงินได้ โดยจะไปเพิ่มยอดเงินในบัญชี ของ address ที่เรียกมา
- 4) สร้างฟังก์ชัน withdraw ที่รับจำนวนที่ต้องการถอนเข้ามา แล้วทำการลดค่ายอดเงินในบัญชี จากนั้นโอนเงินกลับไป smart contract

**** note:** ให้เช็كدัวยว่าบัญชีนั้นมีเงินเพียงพอให้ถอนตามยอดที่ระบุไว้เปล่า

ถอนถอนเงิน (withdraw function) ให้ใช้โค้ดนี้ประกอบเพื่อโอนเงินกลับไปยังบัญชีผู้ถอน

```
(bool success, ) = msg.sender.call{value: amount}("");  
    require(success, "Withdraw failed!!");
```